

PRACTICAL_WEEK-03

Task 3: Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

```
Last login: Sat Aug 22 19:53:08 on ttys000
root@Vishnu:~# cd ~/OSSP
root@Vishnu:~# pwd
/Users/Vishnu/OSSP
root@Vishnu:~#
```

Figure 1: OSSP project directory

```
root@Vishnu:~# cat src/week3_task3.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Before fork()\n");
    printf("Current Process PID = %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("----- Child Process ----- \n");
        printf("Child PID = %d\n", getpid());
        printf("Parent PID = %d\n", getppid());
        printf("State: Running\n");

        printf("Child is sleeping for 5 seconds...\n");
        printf("State: Waiting / Sleeping\n");

        sleep(5);

        printf("Child exiting...\n");
        printf("State: Terminated\n");

        exit(0);
    }
    else
    {
        printf("----- Parent Process ----- \n");
        printf("Parent PID = %d\n", getpid());
        printf("Parent PPID = %d\n", getppid());
        printf("Child PID = %d\n", pid);
        printf("State: Running\n");

        printf("Parent waiting for child...\n");
        printf("State: Waiting\n");

        wait(NULL);

        printf("Child terminated successfully.\n");
        printf("Parent State: Running\n");

        printf("Parent exiting...\n");

        return 0;
    }
}
```

Figure 2: C program using `fork()` to create parent and child processes and PID, PPID and process-state handling using `sleep()` and `wait()`

```
root@Vishnu:~# gcc src/week3_task3.c -o bin/week3_task3
root@Vishnu:~# ./bin/week3_task3
Before fork()
Current Process PID = 74723

----- Parent Process -----
Parent PID = 74723
Parent PPID = 73443
Child PID = 74727
State: Running
Parent waiting for child...
State: Waiting
----- Child Process -----
Child PID = 74727
Parent PID = 74723
State: Running
Child is sleeping for 5 seconds...
State: Waiting / Sleeping
Child exiting...
State: Terminated
Child terminated successfully.
Parent State: Running
Parent exiting...
amrutha@Amruthas-MacBook-Air: OSSP %
```

Figure 3: Successful compilation of the `fork()` process-state program and Execution showing parent and child PIDs, PPIDs and process states



```

graph TD
    END([END])
  
```

```

root@Vishnu:~# git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/parser.c

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        bin/escape_parser
        bin/parser
        bin/quote_parser
        bin/wait_waitpid
        bin/week3_task3
        double_quote_flowchart.txt
        escape_flowchart.txt
        parser_flowchart.txt
        process_flowchart.txt
        quote_flowchart.txt
        src/escape_parser.c
        src/parser.c.save
        src/quote_parser.c
        src/wait_waitpid.c
        src/week3_task3.c
        src/zombie_process.cB
        wait_flowchart.txt
        week3_task3_flowchart.txt

no changes added to commit (use "git add" and/or "git commit -a")
root@Vishnu:~# git add src/week3_task3.c bin/week3_task3 week3_task3_flowchart.txt
root@Vishnu:~# git commit -m "Complete Week 3 Task 3"
[main 6dc4c4a] Complete Week 3 Task 3
 3 files changed, 123 insertions(+)
 create mode 100755 bin/week3_task3
 create mode 100644 src/week3_task3.c
 create mode 100644 week3_task3_flowchart.txt
root@Vishnu:~#
  
```

Figure 4: Terminal flowchart for parent and child process execution using fork(), Git status and Git commit

Task 4: Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
root@Vishnu:~# cd ~/OSSP
root@Vishnu:~# pwd
/Users/Vishnu/OSSP
root@Vishnu:~#
```

Figure 1: OSSP project directory

```
root@Vishnu:~# nano src/process_states.c
root@Vishnu:~# cat src/process_states.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Parent PID = %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child PID = %d\n", getpid());
        printf("Child PPID = %d\n", getppid());

        printf("Child State: Running\n");

        printf("Child entering Waiting/Sleeping state...\n");
        sleep(60);

        printf("Child State: Running again\n");
        printf("Child terminating...\n");

        exit(0);
    }
    else
    {
        printf("Parent waiting for Child PID = %d\n", pid);
        wait(NULL);

        printf("Child terminated.\n");
        printf("Parent terminating.\n");
    }

    return 0;
}
root@Vishnu:~#
```

Figure 2: C program for observing process-state transitions using fork()

```
root@Vishnu:~# ps -p 80190 -o pid,ppid,state,command
PID PPID STAT COMMAND
80190 80189 S+ ./bin/process_states
root@Vishnu:~#
```

Figure 3: Observation of the child process in the waiting/sleeping state using ps

```
Processes: 661 total, 2 running, 659 sleeping, 4306 threads
Load Avg: 2.34, 2.08, 1.89 CPU usage: 4.44% user, 5.48% sys, 90.7% idle SharedLibs: 636M resident, 129M data, 93M linkedit. MemRegions: 66 total, 2832K resident, 3568K private, 1953M shared.
PhysMem: 156 used (1997M wired, 3815M compressor), 476M unused. VM: 328T vsize, 6144M framework vsize, 105009(0) swpins, 316747(0) swapouts. Networks: packets: 24356276/14G in, 4259020/6261M out.
Disks: 22946359/5856 read, 3580662/706 written.
```

Figure 4: Monitoring the process state using the top command

```

Last login: Mon Aug 24 21:15:00 on ttys037
root@Vishnu:~# top -pid 81234
root@Vishnu:~# ls /proc
ls: /proc: No such file or directory
root@Vishnu:~#

```

Figure 5: Verification that the Linux /proc filesystem is not available on macOS

Observation: The /proc virtual filesystem is commonly used in Linux to obtain process information. Since this practical was performed on macOS, /proc is not available. Process information was therefore observed using the macOS-supported ps and top commands.

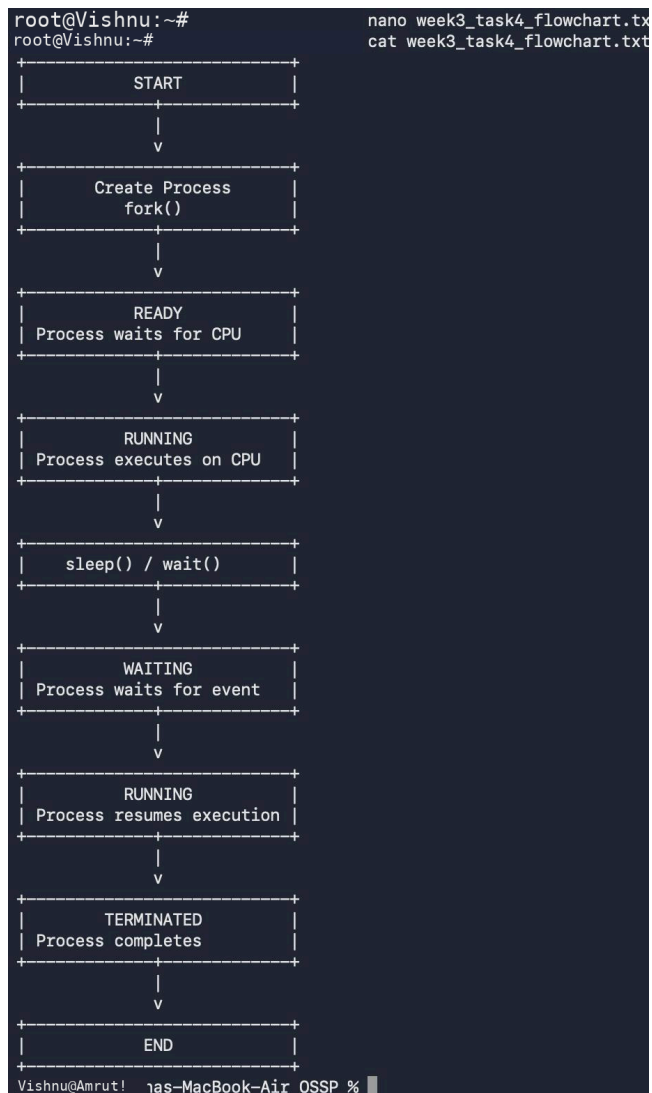


Figure 6: Flowchart showing process-state transitions from Ready to Running, Waiting and Terminated